DevLab 測試區使用說明

公司內部的程式部署測試環境：同事可以把網頁程式放上來試用與展示。

服務清單 連不上怎麼辦 準備你的程式 用 Claude Code 容器化 部署流程 規則 常見情況 常用指令 限制 實例：戰情室

> **第一部分：給所有人** 只使用服務的同事，看完這部分即可

這是什麼

DevLab 是一台放在公司內網的測試主機（`10.125.2.14`），用 Docker 執行各種網頁程式。每個程式都有自己的網址，格式是：

```
http://<程式名稱>.10.125.2.14.sslip.io
```

複製

> **這是測試環境，不是正式系統。**主機可能因為更新或維護而暫停，請不要把它當成唯一的資料來源，重要資料請另外保存。

服務清單

中鴻鋼鐵週報戰情室

http://chs-warroom.10.125.2.14.sslip.io

每週行情、法人、信用交易與籌碼週報。平日 15:30～22:00 自動抓取證交所資料，週五 19:30 擷取 MoneyDJ 圖資。

舊網址 `http://10.125.2.14:5004` 已停用，請更新書籤。

連線測試頁

http://whoami.10.125.2.14.sslip.io

打得開就代表你的電腦能連到 DevLab。連不上其他服務時，先用這頁確認網路。

使用說明（本頁）

http://docs.10.125.2.14.sslip.io

連不上怎麼辦

1. **確認你在公司網路內。**目前只開放以下網段連線，在家或用手機行動網路都連不到：

網段	範圍
`10.125.0.0/21`	10.125.0.1 ~ 10.125.7.254
`10.125.204.0/24`	10.125.204.1 ~ 10.125.204.254
`10.125.241.0/24`	10.125.241.1 ~ 10.125.241.254

   不知道自己的 IP？在「命令提示字元」輸入 `ipconfig`，看「IPv4 位址」。不在上面的範圍內，請找聯絡人開通。

2. **打開連線測試頁。**
   - 測試頁**打得開**、其他服務打不開 → 是該服務的問題，請通知聯絡人。
   - 測試頁也**打不開**（顯示「連線逾時」）→ 網段沒開通，或 DevLab 主機沒在運作（例如重開機後還沒人登入）。
3. **網址打錯或服務不存在**時會看到「404 page not found」，請檢查網址拼字。

聯絡人

DevLab 管理員：邱伯凱

開通網段、回報問題、申請部署程式，都請找管理員。

> **第二部分：給開發者** 想把自己的程式放上 DevLab 的同事

📘 **完整實例：中鴻週報戰情室容器化**

從原本在 Windows 執行的 Python 程式，到容器化、測試、處理公司憑證問題、切換上線的完整過程，附截圖與實際指令輸出。

🛠️ **DevLab 管理員手冊**

主機架構、日常維運、部署審查、備份與退回、公司憑證等已知問題。一般使用者不需要看；部署程式時想了解管理員會檢查什麼，可以參考「3.4 部署新專案」。

準備你的程式

DevLab 上的每個程式都跑在 Docker 容器裡。部署前請確認你的程式符合以下條件：

項目	要求	原因
網頁服務	用 HTTP 提供服務，監聽 `0.0.0.0`，不要只聽 `127.0.0.1`	否則容器外連不進來
連接埠	告訴管理員程式聽哪個埠（例如 8080、3000、5000）	反向代理要知道往哪裡轉
設定與密碼	從環境變數讀取，不要寫死在程式裡	密碼不能進版控或映像檔
資料	要保存的檔案或資料庫放在固定資料夾（例如 `/data`）	容器重建後，只有掛載的資料夾會保留
Log	直接輸出到螢幕（stdout / stderr），不要只寫檔案	管理員才能用 Docker 查看
健康檢查（建議）	提供一個簡單的網址，例如 `/health` 回傳 200	用來判斷程式是否正常
Host 檢查	如果程式會檢查 Host，要允許 `<程式名稱>.10.125.2.14.sslip.io`	否則會回 403

純靜態網頁（HTML、CSS、JavaScript）最簡單：不用 Docker 知識，把檔案交給管理員就能上線。

用 Claude Code 容器化

如果程式是在自己電腦開發的，可以讓自己電腦上的 Claude Code 把程式容器化，再把原始碼交給管理員部署。DevLab 沒有對外開放 Docker 管理介面，所以**無法從你的電腦直接部署到 DevLab**，流程分成兩段：

步驟	在哪裡	做什麼
1. 容器化	你的電腦	用 Claude Code 產生 `Dockerfile`、`.dockerignore`、`compose.yml`、`app.env.example`、`DEPLOY.md`
2. 本機測試（可省略）	你的電腦	有安裝 Docker Desktop 就先 `docker compose up --build` 試跑；沒安裝就跳過，到 DevLab 再測
3. 交付	你 → 管理員	原始碼打包成 zip，**密碼另外私下提供**
4. 部署	DevLab 主機	管理員在 DevLab 上建置映像檔並啟動（見部署流程）

建議交原始碼、在 DevLab 上建置，不要交 `docker save` 匯出的映像檔。公司憑證、Traefik 設定、資源上限都要照 DevLab 的規則調整，之後改版也只要重新建置。

在你的電腦啟動 Claude Code

在專案根目錄開 PowerShell：

```
cd C:\路徑\你的專案
claude
```

複製

建議先按 `Shift+Tab` 切到 plan mode，讓 Claude 先分析並提出計畫，你確認後再讓它修改檔案。

貼上這段提示

把 `<程式名稱>` 換成你的程式名稱（小寫英文、數字、連字號）後整段貼上：

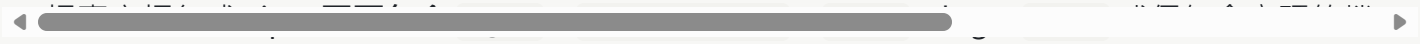
```
請把這個專案容器化，要部署到公司的 DevLab Docker 測試區。
DevLab 規範請先閱讀 http://docs.10.125.2.14.sslip.io 的「準備你的程式」、「部署流程」、「規則」三節。

程式名稱：<程式名稱>（小寫英文、數字、連字號，會成為網址 <程式名稱>.10.125.2.14.sslip.io）
```

```
要求：
1. 先分析專案：語言/框架版本、啟動方式、監聽的連接埠、需要的環境變數、要保存的資料、外部相依（資料庫、SMTP、分
2. 產生 Dockerfile（固定版本號的官方基底映像檔，非 root 執行，盡量用 multi-stage）與 .dockerignore（排除
3. 產生 compose.yml，照 DevLab 範本規則：
   - 服務名稱 app，用 build: . 與 image: devlab/<程式名稱>:dev
   - 不可寫 ports:，改加入外部網路 devlab，加 Traefik labels：
     traefik.enable=true、traefik.docker.network=devlab、
     Host(`<程式名稱>.${LAB_DOMAIN}`)、entrypoints=web、loadbalancer.server.port=<程式埠>
   - 資料庫只放在內部 backend 網路；要保存的資料用 named volume
   - 資源上限 2 CPU / 1 GB、json-file log 上限 10m x 3、restart: unless-stopped
4. 程式要監聽 0.0.0.0、log 輸出到 stdout、設定與密碼從環境變數讀取；最好有 /health。
   必要的程式修改請盡量小，並逐項說明。
5. 密碼不可寫進任何檔案，改產生 app.env.example 列出變數名稱與說明。
6. 程式如果會連外部 HTTPS 網站，請標註出來（公司防火牆會換上 CHS-RootCA 憑證，部署時要處理）。
7. 寫 DEPLOY.md：連接埠、環境變數清單、volume、健康檢查網址、已知風險。
8. 如果本機有 Docker，請建置並啟動測試，確認可以回應 HTTP；沒有就只檢查語法。
最後只回報：新增/修改的檔案、測試結果、需要部署管理員決定的事項。
```

如果 Claude Code 讀不到本頁，請把準備你的程式和規則的內容直接貼給它。

交付給管理員



案。

- 附上 Claude Code 最後的回報與 `DEPLOY.md`，特別是「需要部署管理員決定的事項」。
- 密碼、API 金鑰另外私下提供，不要寄在同一封信或放在 zip 裡。

> Claude Code 改過的程式碼，請自己檢查並確認功能正常後再交付。容器化只應該修改啟動方式、監聽位址和設定讀取方式，不應該改到商業邏輯。

部署流程

部署要在 DevLab 主機上操作。沒有主機權限的同事，請把以下資料交給管理員：

- 程式名稱（小寫英文、數字、連字號，例如 `sales-report`），會成為網址的一部分
- 原始碼或 Docker 映像檔，以及程式監聽的連接埠（可以先用 Claude Code 容器化）
- 需要的環境變數清單（密碼另外私下提供）

- 是否需要資料庫、要保存哪些資料

有主機權限時

1. 用範本建立專案（會自動取得網址並啟動預設頁面）：

   ```
   powershell -ExecutionPolicy Bypass -File D:\lab\apps\new-app.ps1 -Name sales-report
   ```
   複製

   使用現成映像檔時，可以直接指定：

   ```
   powershell -ExecutionPolicy Bypass -File D:\lab\apps\new-app.ps1 -Name api -Port 8080 -Ima
   ```
   複製

2. 修改 `D:\lab\apps\<名稱>\` 底下的設定：

檔案	用途
`compose.yml`	主設定：映像檔或 Dockerfile、資料夾掛載、資料庫
`.env`	程式名稱、連接埠、映像檔、資源上限
`app.env`	傳給程式的環境變數與密碼（不進版控）
`public\`	靜態網頁放這裡就能直接上線

3. 套用設定：

   ```
   docker compose --project-directory D:\lab\apps\sales-report up -d
   ```
   複製

   如果是自己的 Dockerfile（例如用 Claude Code 容器化交來的專案），先建立專案但不要啟動，把原始碼和同事的 `compose.yml` 放進資料夾，填好 `app.env`，再建置並啟動：

   ```
   powershell -ExecutionPolicy Bypass -File D:\lab\apps\new-app.ps1 -Name sales-report -Port
   docker compose --project-directory D:\lab\apps\sales-report up -d --build
   docker compose --project-directory D:\lab\apps\sales-report logs -f
   ```
   複製

4. 打開 `http://sales-report.10.125.2.14.sslip.io` 確認，並請管理員把服務加到本頁的服務清單。

規則（必讀）

1. **不要直接開放連接埠。**`compose.yml` 裡不要寫 `ports:`，一律透過網址連入。防火牆只開放 80 埠，直接開放的連接埠同事也連不到，還可能造成安全漏洞。
2. **密碼放 `app.env`。**不要寫進 `compose.yml`、Dockerfile 或程式碼，也不要提交到 Git。
3. **資料庫只放在專案的內部網路**（範本的 `backend` 網路），不要加入 `devlab` 網路。
4. **資源上限預設是 2 CPU、1 GB 記憶體。**需要更多請先跟管理員確認，這台主機是大家共用的。
5. **不要連正式環境的資料庫做寫入。**DevLab 是測試環境，需要正式資料請用測試資料庫或副本。
6. **不用的專案請通知管理員移除**，釋放資源和網址。

常見情況

程式要寄信

DevLab 內建測試信箱 Mailpit，寄出的信會被攔截保存，**不會真的寄出去**，可以放心測試。程式的 SMTP 設定為：主機 `mailpit`、埠 `1025`、不需帳密與加密（範本的 `app.env` 已預設）。要看寄出的信件，請找管理員。

程式要連外部 HTTPS 網站，出現憑證錯誤

公司防火牆會檢查部分網站的 HTTPS 連線，並換上公司自己簽發的憑證（CHS-RootCA）。Windows 電腦信任這張憑證，但容器預設不信任，所以會出現：

```
CERTIFICATE_VERIFY_FAILED: self-signed certificate in certificate chain
net::ERR_CERT_AUTHORITY_INVALID
```

複製

解法是把公司根憑證加進映像檔，管理員有現成的憑證檔和做法（戰情室就是這樣處理的）。Python 的 `requests`、Node.js、Java、Chromium 各自有自己的信任清單，設定方式不同，請跟管理員討論。

程式拿到的使用者 IP 都是 172.18.0.1

這是 DevLab 的網路架構造成的，程式拿不到使用者的真實 IP，請不要用 IP 做身分驗證或存取控制。需要辨識使用者時，請用帳號登入。

網址只有 HTTP，沒有 HTTPS

目前 DevLab 只提供 HTTP，所以**不要在測試程式裡輸入真實的公司帳號密碼**。需要 HTTPS 的情況請跟管理員討論。

常用指令

以下指令在 DevLab 主機的 PowerShell 執行，`<名稱>` 換成你的程式名稱。

要做什麼	指令
啟動 / 套用設定	`docker compose --project-directory D:\lab\apps\<名稱> up -d`
停止	`docker compose --project-directory D:\lab\apps\<名稱> stop`
看狀態	`docker compose --project-directory D:\lab\apps\<名稱> ps`
看 log（持續顯示）	`docker compose --project-directory D:\lab\apps\<名稱> logs -f --tail 100`
改了 Dockerfile 或程式後重建	`docker compose --project-directory D:\lab\apps\<名稱> up -d --build`
移除程式（保留資料）	`docker compose --project-directory D:\lab\apps\<名稱> down`
移除程式和資料（無法復原）	`docker compose --project-directory D:\lab\apps\<名稱> down -v`

限制與已知問題

- **主機重開機後，要有人登入 Windows**，Docker 和所有服務才會啟動。Windows 更新後服務可能暫停一段時間。
- **只有 HTTP**，沒有 HTTPS。
- **只開放部分網段**，其他網段需要申請開通。
- **網址依賴 sslip.io**：這是外部的免費 DNS 服務，把 `*.10.125.2.14.sslip.io` 解析到 10.125.2.14。如果它停止服務，網址會暫時失效，用 IP 也連不到。

- **備份要自己規劃。**除了戰情室有每日備份，其他程式的資料預設沒有備份，需要的話請跟管理員討論。

最後更新：2026-10-14 · DevLab 測試區（10.125.2.14）

← DevLab 使用說明

實例：中鴻週報戰情室容器化

把一個在 Windows 上直接執行的 Python 程式，轉成 Docker 容器並部署上線的完整紀錄（2026-10-14）。

1. 原本的樣子 2. 盤點 3. 設定檔 4. 建置 5. 測試 6. 憑證問題 7. 切換上線 8. 上線後 檢查清單

成果摘要

	轉換前	轉換後
網址	`http://10.125.2.14:5004`	`http://chs-warroom.10.125.2.14.sslip.io`
執行方式	Windows 工作排程器，開機時以 SYSTEM 身分執行 Python	Docker 容器，非 root 使用者
環境	主機上的 Python 3.11 + 虛擬環境，瀏覽器路徑寫死在使用者資料夾	映像檔內含 Python 3.12、Chromium、中文字型，可在任何 Docker 主機重建
資料	`D:\StockWarRoomV2\data\warroom.db`	Docker volume `chs-warroom_data`，每天自動備份
對外開放	5004 埠，防火牆允許**任何來源**；Host 檢查含 `*`（全部放行）	只經反向代理的 80 埠，限定網段；Host 只接受自己的網址
原始碼	**完全沒改**：Docker 相關檔案都放在另一個資料夾	
停機時間	約 1 分鐘	

1 原本的樣子

戰情室是 Flask 網頁程式，內建排程定時抓證交所資料，並用 Playwright（Chromium）擷取 MoneyDJ 的圖表。原專案資料夾：

```
D:\StockWarRoomV2\
├── run.py                    啟動程式（waitress，預設 5004 埠）
├── requirements-lock.txt     套件版本（Flask、APScheduler、playwright 1.59.0…）
├── warroom\                  主程式：app.py、jobs.py（排程）、pipeline.py、charts.py（截圖）
├── templates\  static\       網頁
├── data\warroom.db           SQLite 資料庫（WAL 模式，約 6 MB）
├── .venv\                    Python 虛擬環境（162 MB）
└── start_server.bat  setup_server.bat
```

複製

它不是用 `start_server.bat` 啟動的，而是**工作排程器**裡的 `\StockWarRoomV2`：開機觸發、以 SYSTEM 身分執行。這是用系統管理員權限查程序來源才發現的：

```
proc 4300 owner=NT AUTHORITY\SYSTEM parent=3816
  parent 3816 svchost.exe cmd=C:\WINDOWS\system32\svchost.exe -k netsvcs -p -s Schedule
task \StockWarRoomV2 state=Running user=SYSTEM trigger=MSFT_TaskBootTrigger
     exec=D:\StockWarRoomV2\.venv\Scripts\python.exe args=-X utf8 D:\StockWarRoomV2\run.py --host 0.
```

複製

為什麼要查這個？如果只在畫面上關掉程式，下次開機工作排程器又會把它啟動，變成原生版和容器版**同時跑排程**，兩邊重複抓資料、各自寫入不同的資料庫。

2 盤點：容器化要注意什麼

讀過 `run.py`、`app.py`、`jobs.py`、`charts.py` 後，整理出影響容器化的地方：

發現	風險	做法
Playwright 用 Chromium 截取中文網頁	容器沒有瀏覽器和中文字型，截圖會變方框	用 Playwright 官方映像檔（版本與 `requirements-lock.txt` 一致），再加裝 Noto CJK 字型
`run.py` 寫死 `PLAYWRIGHT_BROWSERS_PATH=C:\Users\CHSAI\…`	Linux 沒有這個路徑	程式用 `setdefault`，所以在映像檔設好環境變數就會優先使用，**不用改程式**

SQLite 開啟 WAL 模式	放在 Windows 資料夾掛載（bind mount）上可能鎖定或損壞	改放 Docker volume
程式內建 APScheduler 排程	同時跑兩份會重複執行工作	切換時先停用原生版；測試時把副本的排程全部停用
`ALLOWED_HOSTS` 含 `*`	Host 檢查形同虛設	容器版只允許自己的網址
資料庫從環境變數 `WARROOM_DB` 讀取路徑	—	設成 `/data/warroom.db`，指向 volume

3 寫設定檔

所有 Docker 相關檔案都放在 `D:\lab\apps\chs-warroom\`，**原專案資料夾一個檔案都不動**，原作者照常開發，不受影響。

檔案	用途
`Dockerfile`	映像檔內容
`Dockerfile.dockerignore`	只把需要的檔案送進建置（排除 `.venv`、資料庫、測試截圖）
`compose.yml`、`.env`	執行設定：網址、volume、資源上限
`build.ps1`	建置映像檔（原因見第 4 步）
`certs\chs-rootca.crt`	公司根憑證（原因見第 6 步）
`cutover.ps1`、`backup.ps1`	切換與備份

Dockerfile（最終版）

```
FROM mcr.microsoft.com/playwright/python:v1.59.0-noble

# 繁體中文字型：MoneyDJ 圖資截圖需要；libnss3-tools 提供 certutil
RUN apt-get update \
 && apt-get install -y --no-install-recommends fonts-noto-cjk python3-venv libnss3-tools \
```

```
 && rm -rf /var/lib/apt/lists/*

# 公司 HTTPS 檢查根憑證（見第 6 步）
COPY --from=certs chs-rootca.crt /usr/local/share/ca-certificates/chs-rootca.crt
RUN update-ca-certificates \
 && mkdir -p /home/pwuser/.pki/nssdb \
 && certutil -d sql:/home/pwuser/.pki/nssdb -N --empty-password \
 && certutil -d sql:/home/pwuser/.pki/nssdb -A -t "C,," -n CHS-RootCA -i /usr/local/share/ca-certifi
 && chown -R pwuser:pwuser /home/pwuser/.pki

ENV VIRTUAL_ENV=/opt/venv \
    PATH=/opt/venv/bin:$PATH \
    PYTHONUTF8=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    TZ=Asia/Taipei \
    PLAYWRIGHT_BROWSERS_PATH=/ms-playwright \
    REQUESTS_CA_BUNDLE=/etc/ssl/certs/ca-certificates.crt \
    SSL_CERT_FILE=/etc/ssl/certs/ca-certificates.crt \
    WARROOM_DB=/data/warroom.db

WORKDIR /app
COPY requirements-lock.txt .
RUN python3 -m venv $VIRTUAL_ENV && pip install --no-cache-dir -r requirements-lock.txt

COPY run.py ./
COPY warroom ./warroom
COPY templates ./templates
COPY static ./static

RUN mkdir -p /data && chown pwuser:pwuser /data
VOLUME /data
# 不用 root 執行
USER pwuser

EXPOSE 5004
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
  CMD python -c "import json,urllib.request;r=urllib.request.urlopen('http://127.0.0.1:5004/health',

CMD ["python", "run.py", "--host", "0.0.0.0", "--port", "5004"]
```

先複製 `requirements-lock.txt` 安裝套件，再複製程式碼：只改程式碼時，套件那層會沿用快取，重新建置只要幾秒。

compose.yml（重點）

複製

```
services:
  app:
    image: devlab/${APP_NAME}:${APP_VERSION}
    pull_policy: never              # 本機建置，不從網路拉
    restart: unless-stopped
    environment:
      ALLOWED_HOSTS: ${APP_NAME}.${LAB_DOMAIN}   # 只接受自己的網址
    volumes:
      - data:/data                  # SQLite 放 volume
    shm_size: 512m                  # Chromium 需要較大的 /dev/shm
    labels:                         # 交給 Traefik 依網址轉發
      - traefik.enable=true
      - traefik.http.routers.${APP_NAME}.rule=Host(`${APP_NAME}.${LAB_DOMAIN}`)
      - traefik.http.services.${APP_NAME}.loadbalancer.server.port=5004
    networks: [devlab]
    deploy:
      resources:
        limits: { cpus: "2", memory: 2g }
    # 注意：沒有 ports:，不直接開放連接埠

volumes:
  data:
    name: chs-warroom_data
    external: true                  # 手動建立；docker compose down -v 不會誤刪資料
```

4 建置：遇到兩個問題

問題 1：建立 Python 虛擬環境失敗

```
#8 1.202 The virtual environment was not created successfully because ensurepip is not
#8 1.202 available.  On Debian/Ubuntu systems, you need to install the python3-venv
#8 1.202 package using the following command.
```

```
#8 1.202     apt install python3.12-venv
#8 ERROR: process "/bin/sh -c python3 -m venv $VIRTUAL_ENV ..." did not complete successfully:
```

解法

錯誤訊息已經寫出答案：Playwright 映像檔沒有附 `venv` 模組。在 `apt-get install` 加上 `python3-venv`。

問題 2：`docker compose build` 失敗，沒有說明原因

```
> docker compose build
failed to execute bake: exit status 1

> docker buildx bake -f compose.yml     # 直接執行底層指令，才看到真正原因
ERROR: additional privileges requested
```

複製

解法

compose 檔在 `D:\lab\apps\chs-warroom`，原始碼在 `D:\StockWarRoomV2`。Buildx 的安全機制不允許讀取 compose 資料夾以外的檔案。所以把**建置和執行分開**：`build.ps1` 直接用 `docker build` 建置，compose 只負責執行建好的映像檔。

```
docker build -f D:\lab\apps\chs-warroom\Dockerfile `
  --build-context certs=D:\lab\apps\chs-warroom\certs `
  -t devlab/chs-warroom:2.0.0 D:\StockWarRoomV2
```

複製

建置結果：映像檔 3.85 GB（大部分是 Chromium 和字型）。

經驗：工具只回「exit status 1」時，改用底層指令或 `--progress plain` 重跑，通常就能看到真正的錯誤訊息。

5 用資料庫副本測試

原生版還在服務同事，測試不能影響它。做法：

1. 用 SQLite 的**線上備份 API** 複製資料庫，程式不用停，副本內容也一致。不要直接複製 `.db` 檔，WAL 模式下可能少掉還沒寫回的資料。
2. 把**副本裡的排程全部停用**，容器版測試期間不會自己去抓資料。
3. 放進 Docker volume，啟動容器。

```
tables: assets=52, fetch_status=174, holidays=27, jobs=63, legacy_weekly=175, logs=723, ...
active schedules before: 9
active schedules after: 0
integrity: ok
```

複製

測試結果

項目	做法	結果
健康檢查	`/health`	✓ healthy
首頁、靜態檔	透過新網址連線	✓ 200
週報資料	同一週的 `/api/report`，容器版和原生版逐字比對	✓ **完全相同**（143 週）
圖片、CSV 匯出	`/api/image` 、 `/api/export`	✓ 200
Host 檢查	用錯誤的 Host 直接打容器	✓ 403 拒絕
證交所連線	容器內呼叫 API	✓ 200
中文字型	Chromium 截一張中文測試頁	✓ 見下圖
MoneyDJ 連線	容器內連線	✗ **憑證錯誤** → 第 6 步



容器內 Chromium 的中文字型測試：字形正常，沒有方框。

6 憑證問題：公司的 HTTPS 檢查

症狀：容器連不上 MoneyDJ，但 Windows 連得上

```
# Python
SSLCertVerificationError: certificate verify failed: self-signed certificate in certificate cha
# Chromium（Playwright）
Page.goto: net::ERR_CERT_AUTHORITY_INVALID at https://concords.moneydj.com/...
# Windows 主機
host moneydj=200
```

用 `openssl` 看容器收到的憑證鏈，發現 MoneyDJ 的憑證**不是 MoneyDJ 自己的 CA 簽的**，而是公司的：

```
0 s:CN = *.moneydj.com
  i:O = CHS, OU = IT, CN = trust.chs.com.tw          ← 公司防火牆重新簽章
1 s:CN = trust.chs.com.tw
  i:DC = tw, DC = com, DC = chsteel-ad, CN = CHS-RootCA
2 s:CN = CHS-RootCA    （自我簽署的公司根憑證）
```

```
# 對照：證交所沒有被攔截，是正常的 TWCA 憑證
 0 s:CN = www.twse.com.tw
   i:CN = TWCA SSL Certification Authority     Verification: OK
```

公司防火牆會攔截部分網站的 HTTPS，換上公司 CA 簽的憑證。加入公司網域的 Windows 信任這張根憑證，容器裡沒有，所以拒絕連線。

解法：把公司根憑證加進映像檔

1. 從 Windows 憑證存放區（`LocalMachine\Root`）匯出 CHS-RootCA。
2. **先核對指紋**，確認匯出的跟容器看到的是同一張，避免加錯憑證：

```
容器看到的：sha256 Fingerprint=5F:6B:BF:E0:09:29:EB:23:...:AC:BA:A4:AE
匯出的檔案：sha256 Fingerprint=5F:6B:BF:E0:09:29:EB:23:...:AC:BA:A4:AE   ✓ 相同
```
複製

3. 加進**三個**信任清單，因為每個元件各管各的：

元件	信任清單	Dockerfile 裡的做法
系統（openssl、urllib）	`/etc/ssl/certs`	`update-ca-certificates`
Python `requests`	自帶的 certifi，不看系統清單	`REQUESTS_CA_BUNDLE` 指向系統清單
Chromium	自己的 NSS 資料庫	`certutil -A` 加到 `~/.pki/nssdb`

重新建置後，Chromium 在容器內成功擷取 MoneyDJ 的兩張圖資，寫進資料庫：

```
capture 20261009 = [{'kind': 'l4', 'bytes': 150961}, {'kind': 'r2', 'bytes': 63153}]
```
複製

測本週時另外出現「MoneyDJ 顯示的日期區間與要求不符」。先別急著修：這週的結束日 10/16 還沒到，MoneyDJ 會自動把日期改成今天，程式的保護機制因此拒絕存檔。正式排程設在週五 19:30，那時結束日就是當天，不會有這個問題。改測上一週就成功了。

經驗：遇到錯誤時先確認原本的版本在同樣條件下會不會也這樣，不要把原本就有的行為誤當成容器化造成的問題。

7 切換上線

選在平日排程（15:30 起）之前的 11:04 切換，並寫成腳本 `cutover.ps1`，讓每一步都可以重複執行，也可以退回：

1. 停止並停用工作排程器的 `\StockWarRoomV2`，確認 5004 埠已經沒有程式在聽
2. 用線上備份 API 做**正式資料庫**的快照，並做完整性檢查
3. 把快照放進 volume，取代測試副本（排程維持原本的啟用設定）
4. 啟動容器，確認 healthy、排程啟用數量與快照一致
5. 停用舊網址 5004 的防火牆規則

退回只要一行：`cutover.ps1 -Rollback`，會把容器期間的資料匯出，再重新啟用原生版。

第一次執行在第 3 步中斷

```
== 1. 停止並停用原生版排程工作 ==
原生版已停止，排程工作已停用
== 2. 資料庫快照 ==
快照：...\backup\native-20261014-110412.db  完整性=ok  週數=142  啟用排程=9
== 3. 匯入 Docker volume ==
ERROR:  Container chs-warroom-app-1 Stopping
```

複製

原因與解法

`Container ... Stopping` 只是正常的進度訊息，但 docker 把它寫到錯誤輸出（stderr），PowerShell 5.1 在嚴格錯誤模式（`$ErrorActionPreference = 'Stop'`）下就當成錯誤中止了。改成 docker 指令一律在寬鬆模式下執行，再檢查結束代碼 `$LASTEXITCODE`。因為腳本可以重複執行，修正後直接重跑。中斷期間服務停了不到 1 分鐘，資料沒有任何影響。

```
== 1. 停止並停用原生版排程工作 ==
目前狀態：Disabled
== 2. 資料庫快照 ==
快照：...\backup\native-20261014-110453.db  完整性=ok  週數=142  啟用排程=9
```

複製

```
== 3. 匯入 Docker volume ==
== 4. 啟動容器並驗證 ==
== 5. 停用舊網址 5004 的防火牆規則（改用新網址） ==
防火牆規則 StockWarRoomV2 Enabled=False
切換完成：http://chs-warroom.10.125.2.14.sslip.io  排程執行中，啟用 9 項
```

切換後的獨立驗證

腳本說成功還不夠，另外逐項比對容器和原本的資料庫：

項目	原生版資料庫	容器版
週報摘要	142	142 ✓
原始資料	1,355	1,355 ✓
執行紀錄	724	724 ✓
10/9 週圖資時間	2026-10-09 22:00:40	相同 ✓（測試時擷取的圖已被正式資料取代）
原生程式	0 個程序，5004 埠無人監聽 ✓	
排程	9/9 啟用，下次執行：今天 15:30 股價行情 ✓	

8 上線後



新網址的戰情室首頁（10/5 ~ 10/9 這一週）。



L4. 券商分點主力進出狀況

MoneyDJ · 指定週區間

中鴻(2014) 券商分點-進出明細

單位：張　最後更新日：2026/10/08

請選擇　自設區間：從 115 年 10 月 5 日 ～ 115 年 10 月 9 日 查詢

買超					賣超				
買超券商	買進	賣出	買超	佔成交比重	賣超券商	買進	賣出	賣超	佔成交比重
國泰-敦南	437	141	296	5.73%	凱基-士林	1	350	349	6.76%
新加坡商瑞銀	294	115	179	3.46%	凱基-台北	83	226	143	2.77%
新光-高雄	173	2	171	3.31%	玉山-大里	2	120	118	2.28%
美商高盛	352	193	159	3.08%	富邦-嘉義	0	106	106	2.05%
新光	185	34	151	2.92%	元大-竹北	0	103	103	1.99%
富邦-岡山	110	0	110	2.13%	華南永昌-岡山	21	83	62	1.2%
凱基-高美館	83	8	75	1.45%	摩根大通	160	218	58	1.12%
凱基-高雄	75	1	74	1.43%	台灣摩根士丹利	181	239	58	1.12%
港商野村	68	0	68	1.32%	富銀-民權	5	62	57	1.1%
元大-永和	61	0	61	1.18%	統一-台中	1	54	53	1.03%
元大-土城永寧	50	0	50	0.97%	元大-高華	28	80	52	1.01%
富邦-新竹	51	4	47	0.91%	富邦-公益	6	57	51	0.99%
台新	56	9	47	0.91%	富邦-高雄	1	51	50	0.97%
元大-鳳中	44	0	44	0.85%	兆豐-鹿港	0	49	49	0.95%
群益金鼎-嘉義	40	0	40	0.77%	富邦-新店	1	47	46	0.89%
合計買超張數			1,572		合計賣超張數				1,354
平均買超成本			16.95		平均賣超成本				16.94

[註1] 合計買超或賣超，為上述家數合計。
[註2] 平均買超或賣超成本，為上述家數合計買賣超金額/上述家數合計買賣超張數。

券商分點要求區間 20261005–20261009 · 擷取／匯入時間：2026-10-09T22:00:40+08:00

R2. 籌碼分佈

MoneyDJ · 籌碼快照

中鴻(2014)籌碼流向

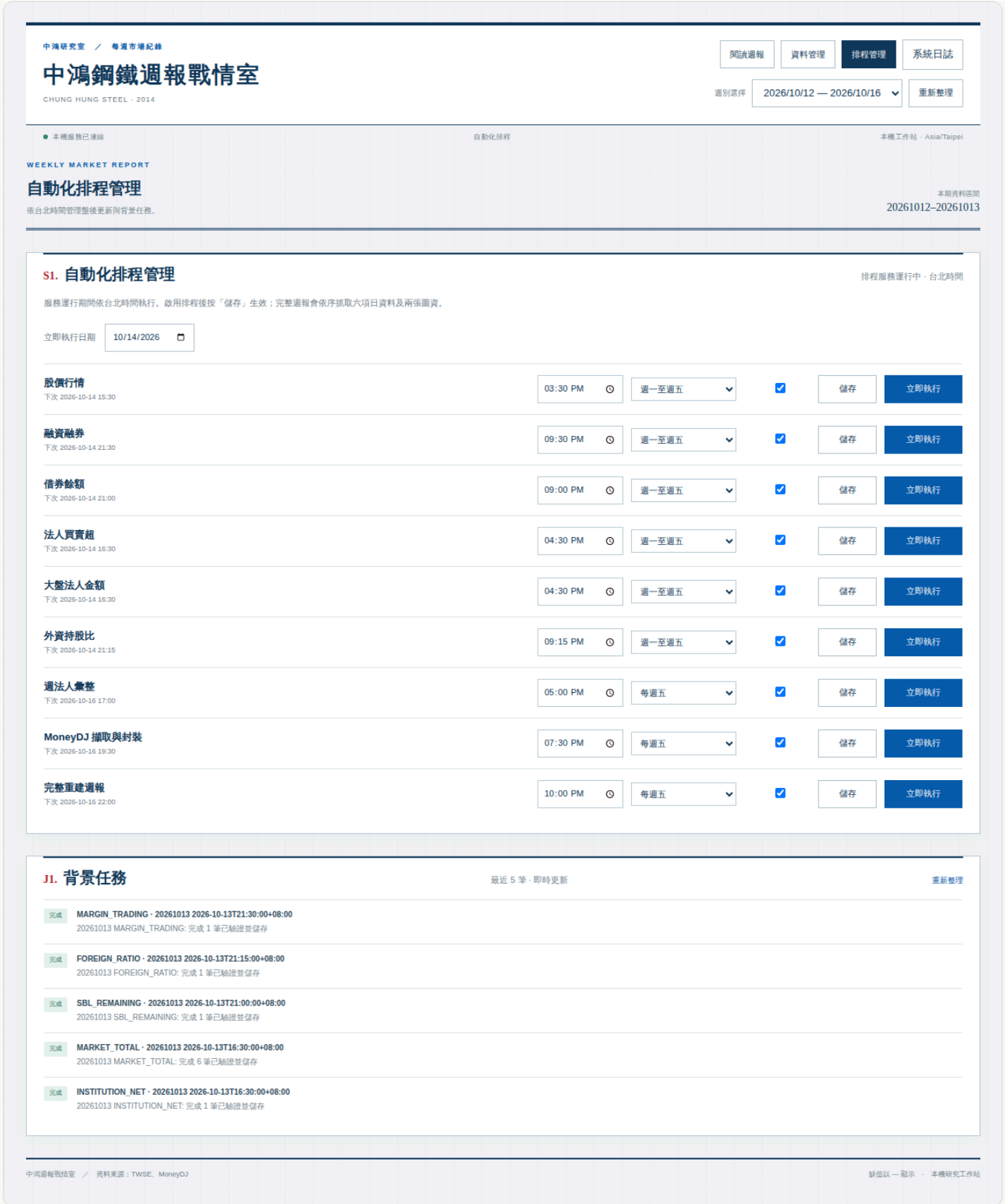
日期：10/08

	最新張數	當日(10/08)增減	佔股本比例
董監持股	582,673		40.59%
外資持股	105,214	378	7.32%
投信持股	4	N/A	0.00%
自營商持股	64,412	63	4.49%
融資餘額	521	-1	0.04%
融券餘額	0	0	

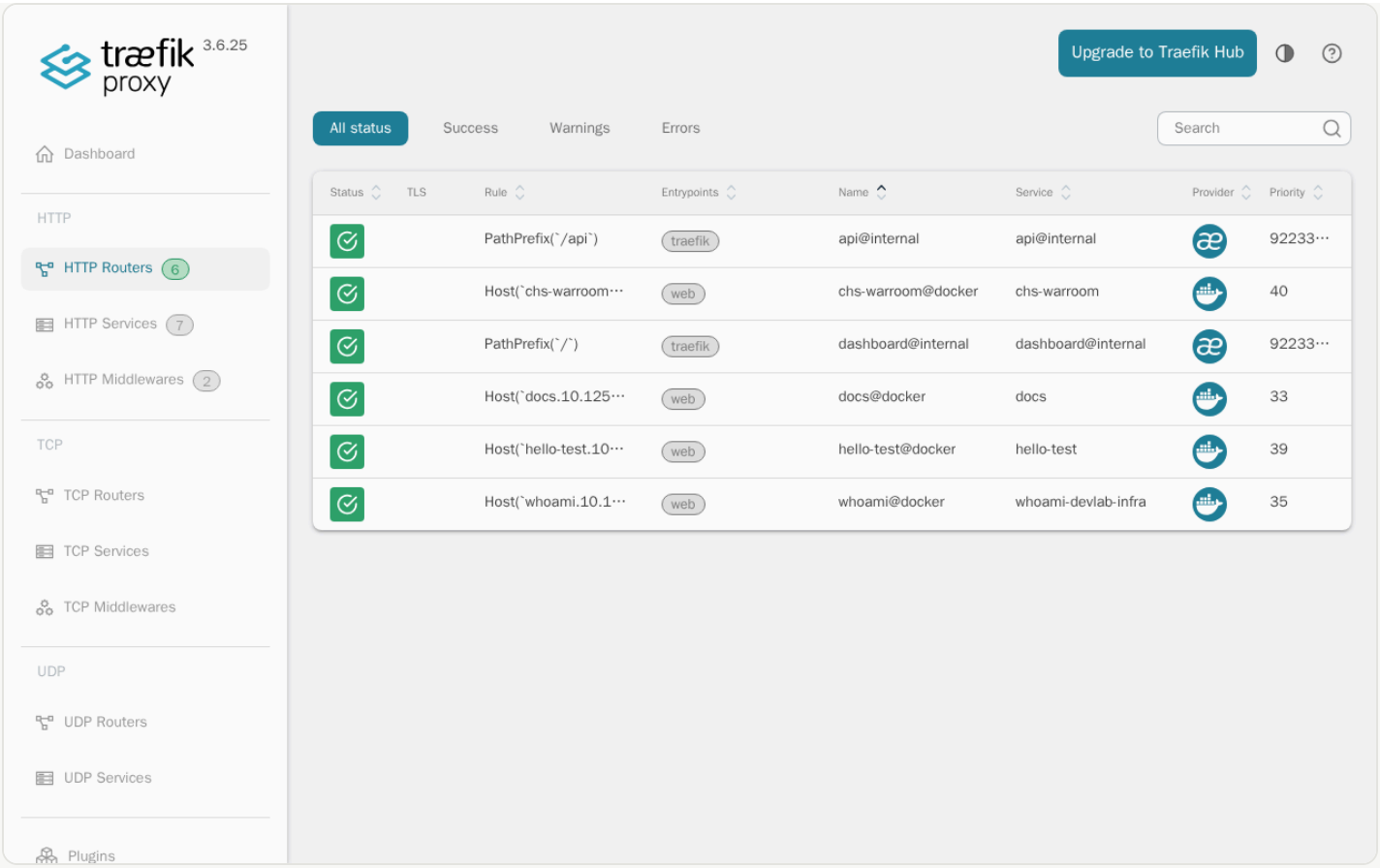
說明：由於董監可能是外資，故股本比例總和有可能超過100%

擷取當下的籌碼流向快照；不是指定歷史週區間 · 擷取／匯入時間：2026-10-09T22:00:41+08:00

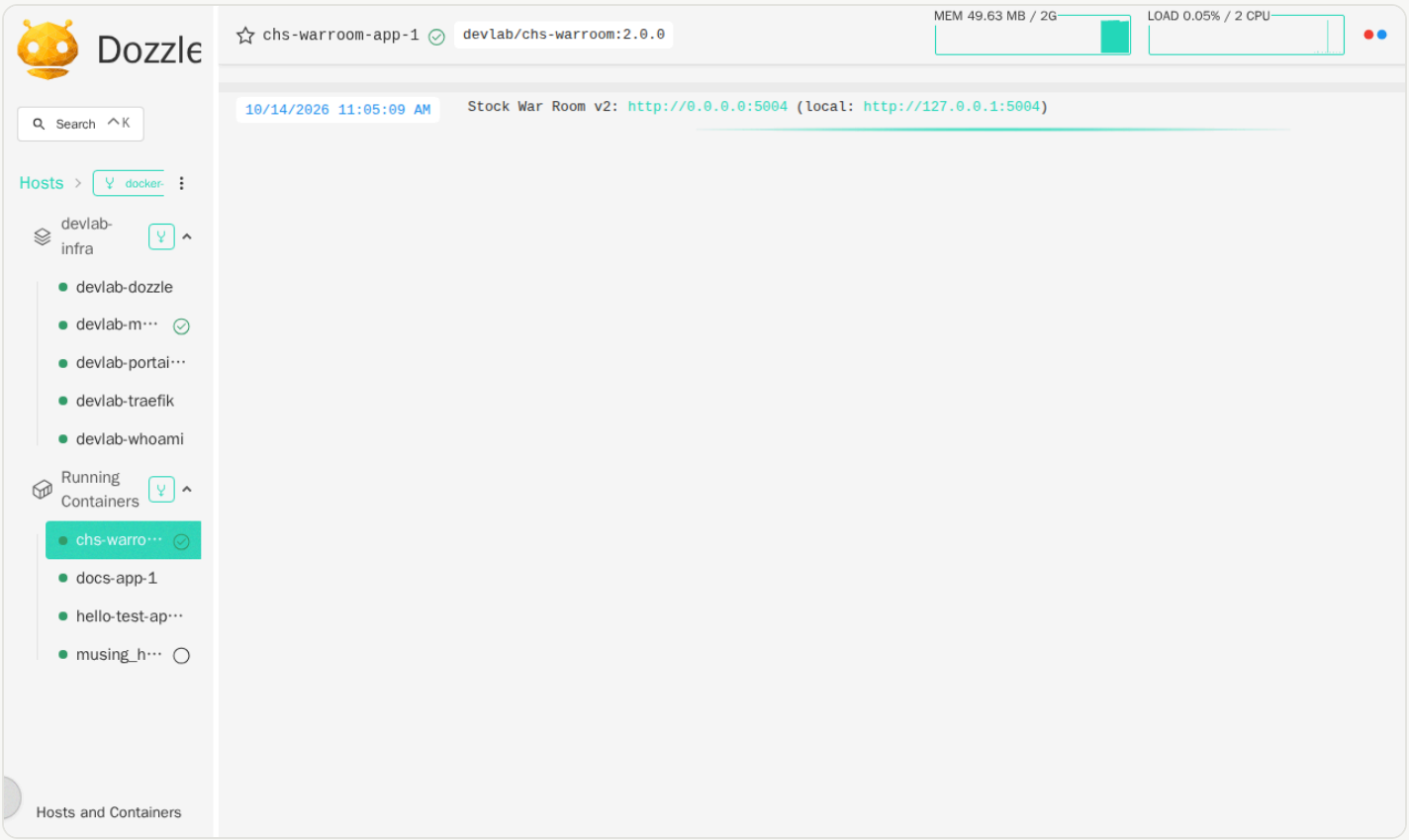
MoneyDJ 圖資：L4 券商分點、R2 籌碼分佈。這是第 6 步解決憑證問題後，在容器裡用 Chromium 擷取的。



排程管理頁：9 項排程都是啟用狀態，下次執行時間正確（台北時間）。



Traefik 儀表板（管理員專用）：`chs-warroom@docker` 依 compose.yml 的 labels 自動建立路由，不用手動設定反向代理。



Dozzle（管理員專用）：即時 log，以及記憶體（約 50 MB / 上限 2 GB）和 CPU 用量。

日常維運

要做什麼	方式
程式改版	`build.ps1` 重新建置 → `docker compose up -d`，避開 15:30～22:00
備份	每天 23:30 自動備份到 `D:\lab\backups\chs-warroom`，保留 30 天
退回原生版	`cutover.ps1 -Rollback`

給你的檢查清單

把自己的程式容器化時，可以照這個實例逐項確認：

1. **找出程式怎麼啟動的。**工作排程器、Windows 服務、啟動資料夾都要查，切換時要一起停用，避免兩份同時跑。
2. **找出寫死的路徑。**例如 `C:\Users\...`、`D:\...`，改成從環境變數讀取。
3. **原始碼和 Docker 設定分開放**，不影響原本的開發流程。
4. **映像檔版本要固定**，並跟程式的套件版本一致（例如 Playwright 1.59.0）。
5. **資料放 volume**；SQLite 尤其不要放 Windows 資料夾掛載。
6. **用資料副本測試**，並停用副本裡會對外動作的功能（排程、寄信）。
7. **跟原本的版本比對輸出**，不要只看「網頁打得開」。
8. **連外部 HTTPS 出現憑證錯誤時**，先查是不是公司的 HTTPS 檢查，再把根憑證加進每個元件的信任清單。
9. **切換寫成可重複執行、可退回的腳本**，選在不影響使用者的時間。
10. **切換後獨立驗證**，並設定好備份。

需要協助請找管理員，見使用說明的聯絡人。

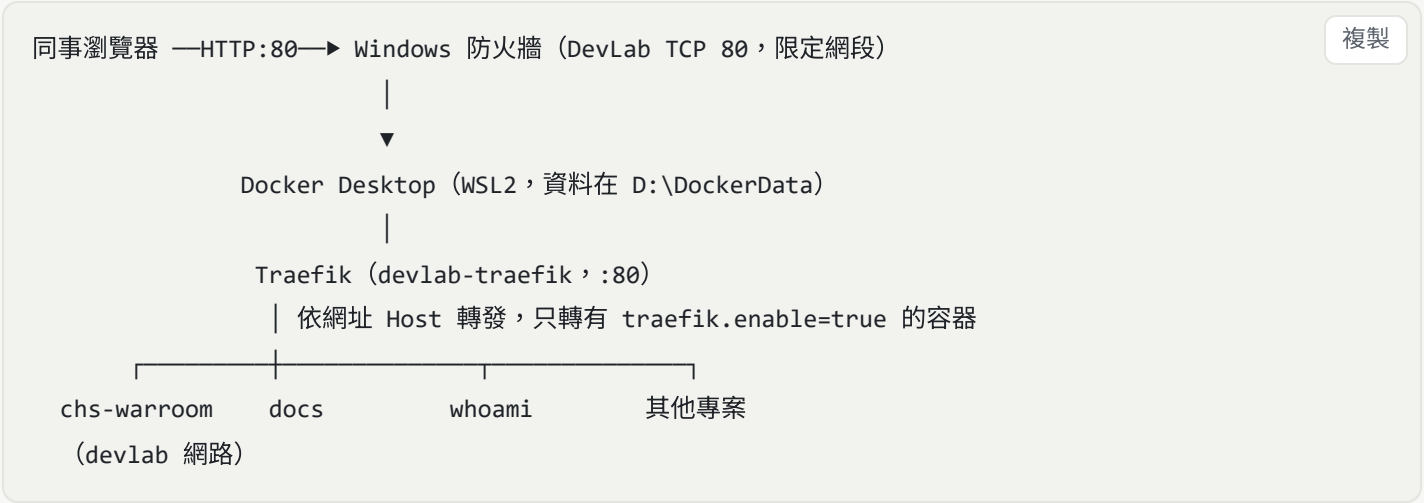
← DevLab 使用說明

DevLab 測試區管理員手冊

最後更新：2026-10-14　適用主機：ADM-014（10.125.2.14）　管理員：邱伯凱
給同事看的使用說明：http://docs.10.125.2.14.sslip.io（原始檔 `D:\lab\apps\docs\public\index.html`）
網頁版：http://docs.10.125.2.14.sslip.io/admin.html（公開給所有能連 DevLab 的同事，2026-10-14 決定）。
本檔是原始檔，修改後執行 `python D:\lab\apps\docs\build-admin.py` 重新產生網頁版。**不要在本手冊寫入任何密碼或金鑰。**

1. 架構總覽　2. 管理工具（僅限本機）　3. 日常維運　4. 中鴻鋼鐵週報戰情室
5. 已知問題與解法　6. 安裝紀錄（重建主機時參考）　7. 待決事項

1. 架構總覽



元件	版本 / 設定
WSL	3.0.3（從 GitHub 下載 MSI 安裝；公司網路封鎖 GitHub API，`wsl --update` 會 403）
Docker Desktop	4.95.0，Engine 29.8.2，資料 `D:\DockerData`

元件	版本 / 設定
WSL 資源上限	`C:\Users\CHSAI\.wslconfig`：40 GB 記憶體、16 處理器、8 GB swap
網域	`*.10.125.2.14.sslip.io`（設定在 `D:\lab\infra\.env` 的 `LAB_DOMAIN`）
主機身分	WORKGROUP，網路設定檔「公用」（Public）

目錄

路徑	內容
`D:\lab\setup\`	安裝腳本 01 ~ 05、安裝紀錄、WSL 安裝檔
`D:\lab\infra\`	共用服務 compose（Traefik、Portainer、Dozzle、Mailpit、whoami）
`D:\lab\apps\_template\`	專案範本
`D:\lab\apps\new-app.ps1`	建立新專案
`D:\lab\apps\<名稱>\`	各專案
`D:\lab\backups\`	備份（`docs\` 為說明頁與本手冊修改前的備份）

2. 管理工具（僅限本機）

這些工具都只綁定 `127.0.0.1`，區網連不到。原因：Docker Desktop 會把來源 IP 換成 `172.18.0.1`，Traefik 無法用 IP 白名單保護，所以只能靠「不對外」來保護。

工具	網址	用途	注意
Traefik 儀表板	http://127.0.0.1:8088/dashboard/	看路由、哪些網址有效	

工具	網址	用途	注意
Portainer	https://127.0.0.1:9443	圖形化管理容器	**等於主機管理員權限**；首次啟動 5 分鐘內要設密碼，逾時執行 `docker restart devlab-portainer`
Dozzle	http://127.0.0.1:9999	即時看所有容器 log	log 可能含機敏資訊
Mailpit	http://127.0.0.1:8025	看程式寄出的測試信	SMTP：容器內 `mailpit:1025`，本機 `127.0.0.1:1025`

3. 日常維運

3.1 重開機後

Docker Desktop **要有人登入 Windows 才會啟動**（選擇的方案 B）。重開機後：

1. 登入 Windows（CHSAI），等 Docker Desktop 顯示 Engine running
2. 所有容器設定為 `restart: unless-stopped`，會自動啟動
3. 確認：

```
docker ps --format "table {{.Names}}\t{{.Status}}"
```
複製

4. 完整驗證：`powershell -ExecutionPolicy Bypass -File D:\lab\setup\04-verify.ps1`

3.2 Docker Desktop 更新後（必做）

更新可能重新建立「Docker Desktop Backend」全開防火牆規則（允許所有連接埠、任何來源），以系統管理員執行：

```
powershell -ExecutionPolicy Bypass -File D:\lab\setup\05-disable-docker-backend-rules.ps1
```
複製

3.3 新增可連線的網段

同事連不上、`ipconfig` 顯示的 IP 不在允許範圍時。以系統管理員執行，**必須列出全部網段**（腳本會重建規則，沒列到的會被移除）：

```
D:\lab\setup\03-open-firewall.ps1 -RemoteAddress 'LocalSubnet','10.125.204.0/24','10.125.241.
```

複製

目前允許：`LocalSubnet`（10.125.0.0/21）、`10.125.204.0/24`（2026-10-15 開通）、`10.125.241.0/24`。新增後記得同步更新使用說明的「連不上怎麼辦」。

3.4 部署新專案

```
powershell -ExecutionPolicy Bypass -File D:\lab\apps\new-app.ps1 -Name <名稱> [-Port 8080] [-
```

複製

同事自帶 Dockerfile 的專案（用 Claude Code 容器化）

同事在自己電腦用 Claude Code 容器化，流程與提示範本在使用說明的「用 Claude Code 容器化」（`id="containerize"`）。DevLab 不對外開放 Docker API，也沒有 registry，所以一律由管理員在本機建置：

1. 收到 zip、`DEPLOY.md` 與 Claude Code 的回報；密碼應另外私下提供，zip 裡若有密碼檔請退回
2. 解壓到暫存資料夾先檢查（不要直接在裡面執行任何腳本）：Dockerfile 的基底映像檔與版本、`compose.yml` 是否符合下方檢查清單、`.dockerignore` 是否排除 `.git`、`node_modules`、`venv`、密碼檔
3. 建立專案並放入原始碼：

   ```
   powershell -ExecutionPolicy Bypass -File D:\lab\apps\new-app.ps1 -Name <名稱> -Port <程式埠
   # 原始碼與同事的 compose.yml 放進 D:\lab\apps\<名稱>\（取代範本的 compose.yml），填好 app.env
   docker compose --project-directory D:\lab\apps\<名稱> up -d --build
   docker compose --project-directory D:\lab\apps\<名稱> logs -f
   ```

   複製

   `new-app.ps1` 產生的 `.env` 已含 `APP_NAME`、`LAB_DOMAIN`、`APP_PORT`，同事的 compose.yml 可直接引用
4. 程式會連外部 HTTPS 時，依 5.1 加入 CHS-RootCA
5. 確認網址可開，再把服務加進使用說明與本手冊

不收 `docker save` 匯出的映像檔：無法檢查內容，公司憑證與設定也要在本機調整，改版時重新建置即可。

部署前檢查：

- ☐ `compose.yml` 沒有 `ports:`（資料庫尤其不能有）
- ☐ 密碼在 `app.env`，沒有寫在 `compose.yml` 或 Dockerfile
- ☐ 資料庫只在 `backend` 網路
- ☐ 映像檔版本固定（不要用 `latest`）
- ☐ 程式需要連外部 HTTPS 時，已處理公司根憑證（見 5.1）
- ☐ 上線後把服務加進使用說明 `D:\lab\apps\docs\public\index.html` 的「服務清單」

3.5 更新使用說明

直接編輯 `D:\lab\apps\docs\public\index.html`，存檔即生效（nginx 直接讀取資料夾，不用重啟）。聯絡人姓名在 `id="contact-name"` 那一行（目前：邱伯凱）。

- 修改前的備份放 `D:\lab\backups\docs\`，**不要放在 `public\` 裡**：該資料夾的所有檔案都會公開在網站上
- 新增服務、網段或流程時，使用說明與本手冊兩份都要更新
- 目前章節：第一部分（服務清單、連不上怎麼辦、聯絡人）、第二部分（準備你的程式、用 Claude Code 容器化、部署流程、規則、常見情況、常用指令、限制）；另有實例頁 `case-warroom.html`

3.6 清理

```
docker system df                    # 看用量
docker image prune                  # 刪除沒有標籤的舊映像檔
docker builder prune                # 刪除建置快取
```

複製

不要用 `docker system prune -a --volumes`，會刪除停止中專案的資料。

4. 中鴻鋼鐵週報戰情室

項目	內容
網址	http://chs-warroom.10.125.2.14.sslip.io
原始碼	`D:\StockWarRoomV2`（Flask + APScheduler + Playwright，原專案資料夾未放任何 Docker 檔案）
Docker 設定	`D:\lab\apps\chs-warroom\`
映像檔	`devlab/chs-warroom:2.0.0`（Playwright 1.59 官方映像檔 + Noto CJK 字型 + CHS-RootCA）
資料	Docker volume `chs-warroom_data`（`external: true`，`docker compose down -v` 不會刪除）
排程	平日 15:30～21:30 抓證交所資料；週五 17:00、19:30（MoneyDJ 圖資）、22:00（重建週報）
切換日期	2026-10-14 11:04 從原生版切換到容器

4.1 程式更新後重新部署

```
powershell -ExecutionPolicy Bypass -File D:\lab\apps\chs-warroom\build.ps1
docker compose --project-directory D:\lab\apps\chs-warroom up -d
```

複製

- 要避開平日 15:30～22:00（排程時間）
- 版本號在 `.env` 的 `APP_VERSION`，大改版時調高，方便退回舊映像檔
- 為什麼建置要獨立成 `build.ps1`：compose 的 bake 不允許讀取 compose 資料夾以外的原始碼（`additional privileges requested`）

4.2 退回原生版

以系統管理員執行：

```
# 只退回，容器期間的新資料匯出到 backup\from-docker-*.db
powershell -ExecutionPolicy Bypass -File D:\lab\apps\chs-warroom\cutover.ps1 -Rollback
```

```
# 退回並把容器期間的資料帶回原生版（會先備份原生資料庫）
powershell -ExecutionPolicy Bypass -File D:\lab\apps\chs-warroom\cutover.ps1 -Rollback -RestoreDataT
```

退回會重新啟用工作排程器 `\StockWarRoomV2`（開機觸發、SYSTEM 身分）與防火牆規則 `StockWarRoomV2`（5004）。

絕對不要讓原生版和容器版同時執行：兩邊的排程會重複抓資料、各自寫入不同的資料庫。

4.3 備份與還原

項目	內容
排程工作	`\DevLab\chs-warroom-backup`，每天 23:30，以 CHSAI 互動身分執行，錯過會在登入後補做
腳本	`D:\lab\apps\chs-warroom\backup.ps1`（線上備份；`-Offline` 直接讀 volume）
位置	`D:\lab\backups\chs-warroom\`，保留 30 天
紀錄	`backup.log`（`OK` / `DEL` / `FAIL`）
切換前快照	`D:\lab\apps\chs-warroom\backup\native-20261014-*.db`

還原步驟見 `D:\lab\backups\chs-warroom\README.txt`。

待辦：備份與正式資料都在 D 槽，建議定期複製到網路磁碟或 NAS。

5. 已知問題與解法

5.1 公司 HTTPS 檢查（CHS-RootCA）

公司防火牆會攔截部分網站（已確認 MoneyDJ；證交所沒有）的 HTTPS，以 `trust.chs.com.tw`（由 CHS-RootCA 簽發）重新簽章。容器預設不信任。

項目	內容
憑證檔	`D:\lab\apps\chs-warroom\certs\chs-rootca.crt`（從 Windows `LocalMachine\Root` 匯出）
SHA-256	`5F6BBFE00929EB237206645FF947497199601CC7AD9A7E286576AB15ACBAA4AE`
有效期限	2033-08-02

需要加入的信任清單：

執行環境	做法
Debian/Ubuntu 系統（openssl、curl、Python urllib）	複製到 `/usr/local/share/ca-certificates/` 後執行 `update-ca-certificates`
Python `requests`	`REQUESTS_CA_BUNDLE=/etc/ssl/certs/ca-certificates.crt`
Chromium / Playwright	`certutil -d sql:$HOME/.pki/nssdb -A -t "C,," -n CHS-RootCA -i <crt>`（需 `libnss3-tools`）
Node.js	`NODE_EXTRA_CA_CERTS=<crt>`
Java	`keytool -importcert -cacerts -alias chs-rootca -file <crt>`

做法範例見 `D:\lab\apps\chs-warroom\Dockerfile`。

5.2 Traefik 版本

Traefik **v3.5 與 Docker Engine 29 不相容**（provider 回 `400 Bad Request`，所有網址 404），必須使用 v3.6 以上。升級 Docker Desktop 後若所有網址突然 404，先查 `docker logs devlab-traefik`。

5.3 PowerShell 5.1 與 docker 指令

在 `$ErrorActionPreference = 'Stop'` 的腳本中，docker 寫到 stderr 的正常進度訊息會被當成錯誤而中止（戰情室切換時發生過）。腳本裡的 docker 指令請用 `Continue` 模式執行並檢查 `$LASTEXITCODE`，參考 `cutover.ps1` 的 `Docker` 函式。

5.4 SQLite WAL 模式

使用 WAL 模式的 SQLite 資料庫不要放在 Windows 資料夾掛載（bind mount），請用 Docker volume，避免鎖定或毀損。

5.5 下載被擋

`wsl --install`、`wsl --update` 透過 GitHub API 查版本，公司網路回 403。github.com 本身可以連，直接下載 Release 檔即可。

6. 安裝紀錄（重建主機時參考）

依序執行 `D:\lab\setup\`：

腳本	權限	用途
`01-enable-wsl.ps1`	管理員	啟用 WSL2、寫入 `.wslconfig`（之後要重開機）
`02-install-docker-desktop.ps1`	管理員	安裝 Docker Desktop（驗證簽章、資料放 D 槽、加入 docker-users）
`03-open-firewall.ps1`	管理員	開放 80 埠給指定網段
`04-verify.ps1`	一般	驗證 8 項
`05-disable-docker-backend-rules.ps1`	管理員	停用 Docker Desktop 全開規則

接著：`docker compose -f D:\lab\infra\compose.yml up -d`，再逐一啟動 `D:\lab\apps\` 底下的專案。

7. 待決事項

- ☑ 使用說明的聯絡人：邱伯凱（2026-10-14 填入）
- ☐ 戰情室備份異地保存（網路磁碟或 NAS）
- ☐ 是否設定 Windows 自動登入（需 IT 同意），讓重開機後服務自動恢復
- ☐ 是否向 IT 申請公司 DNS 萬用網域，取代 sslip.io
- ☐ HTTPS（需公司 CA 簽發憑證）
- ☐ `hello-test` 測試專案可刪除：`docker compose --project-directory D:\lab\apps\hello-test down -v; Remove-Item D:\lab\apps\hello-test -Recurse`

最後更新：2026-10-14 · 回 DevLab 使用說明